

lab2Li

```
library(tidyr)
library(readr)
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v purrr      1.2.2
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(dplyr)
```

```
atraso <- function(input, put){
  input |>
    filter(!is.na(ARRIVAL_DELAY)) |>
    summarise(
      n_voos = n(),
      n_atrasos = sum(ARRIVAL_DELAY > 10, na.rm = TRUE))|>
    mutate(
      estatistica_suficiente = n_atrasos / n_voos
    )}

paia = read_csv_chunked('flights.csv.zip',
  callback=DataFrameCallback$new(atraso),
  chunk_size = 1e5)
```

```
-- Column specification -----
cols(
  .default = col_double(),
  AIRLINE = col_character(),
  TAIL_NUMBER = col_character(),
  ORIGIN_AIRPORT = col_character(),
  DESTINATION_AIRPORT = col_character(),
  SCHEDULED_DEPARTURE = col_character(),
  DEPARTURE_TIME = col_character(),
  WHEELS_OFF = col_character(),
  WHEELS_ON = col_character(),
  SCHEDULED_ARRIVAL = col_character(),
  ARRIVAL_TIME = col_character(),
  CANCELLATION_REASON = col_character()
)
i Use `spec()` for the full column specifications.
```

Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dplyr`:

Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: AA, DL, UA e US; Remova observações que tenham valores faltantes em campos de interesse; Agrupe o conjunto de dados resultante de acordo com: dia, mês e cia. aérea; Para cada grupo em `b.`, determine as estatísticas suficientes apontadas no item 1. e os retorne como um objeto da classe `tibble`; A função deve receber apenas dois argumentos: `input`: o conjunto de dados (referente ao lote em questão); `pos`: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir na função, este argumento não será empregado internamente. É importante observar que, sem a definição deste argumento, será impossível fazer a leitura por partes.

```
getStats = function(input, pos) {
  input|>
    filter(AIRLINE %in% c("AA", "DL", "UA", "US")) |>
    filter(!is.na(ARRIVAL_DELAY))|>
    group_by(DAY, MONTH, AIRLINE)|>
    summarize(
      n_voos = n(),
      n_total_atraso = sum(ARRIVAL_DELAY > 10, na.rm = TRUE),
      .groups = "drop"
    )|>
    mutate(
      estatistica_suficiente = (n_total_atraso/n_voos)
```

```
)
}
```

```
in2 = read_csv_chunked('flights.csv.zip',
  callback=DataFrameCallback$new(getStats),
  chunk_size = 1e5)
```

```
-- Column specification -----
cols(
  .default = col_double(),
  AIRLINE = col_character(),
  TAIL_NUMBER = col_character(),
  ORIGIN_AIRPORT = col_character(),
  DESTINATION_AIRPORT = col_character(),
  SCHEDULED_DEPARTURE = col_character(),
  DEPARTURE_TIME = col_character(),
  WHEELS_OFF = col_character(),
  WHEELS_ON = col_character(),
  SCHEDULED_ARRIVAL = col_character(),
  ARRIVAL_TIME = col_character(),
  CANCELLATION_REASON = col_character()
)
i Use `spec()` for the full column specifications.
```

Utilize alguma função `readr::read_***_chunked` para importar o arquivo `flights.csv.zip`.

Configure o tamanho do lote (chunk) para 100 mil registros; Configure a função de callback para instanciar DataFrames aplicando a função `getStats` criada acima; Configure o argumento `col_types` de forma que ele leia, diretamente do arquivo, apenas as colunas de interesse (veja nota de aula para identificar como realizar esta tarefa)

```
lalala = read_csv_chunked('flights.csv.zip',
  callback = DataFrameCallback$new(getStats),
  chunk_size = 1e5,
  col_types = cols_only(
    MONTH = col_integer(),
    DAY = col_integer(),
    AIRLINE = col_character(),
    ARRIVAL_DELAY = col_double()
  ))
lalala
```

```
# A tibble: 1,478 x 6
  DAY MONTH AIRLINE n_voos n_total_atraso estatistica_suficiente
  <int> <int> <chr>    <int>      <int>          <dbl>
1     1     1     1 AA      1379        514          0.373
2     2     1     1 DL      1558        132          0.0847
3     3     1     1 UA      1269        367          0.289
4     4     1     1 US      1028        201          0.196
5     5     2     1 AA      1508        649          0.430
6     6     2     1 DL      2261        406          0.180
7     7     2     1 UA      1411        493          0.349
8     8     2     1 US      1174        229          0.195
9     9     3     1 AA      1425        767          0.538
10    10     3     1 DL      2021        702          0.347
# i 1,468 more rows
```

Crie uma função chamada `computeStats` que:

Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea); Retorne as informações em um tibble contendo apenas as seguintes colunas: `Cia`: sigla da companhia aérea; `Data`: data, no formato AAAA-MM-DD (dica: utilize o comando `as.Date`); `Perc`: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo [0,1]

```
computeStats <- function(dados){
  dados|>
    mutate(
      Data = as.Date(paste(2015, MONTH, DAY, sep = "-")),
      Cia = AIRLINE,
      Perc = n_total_atraso/n_voos
    )|>
    select(Data, Cia, Perc)}

aleluia <- computeStats(lalala)
```

Produza um mapa de calor em formato de calendário para cada Cia. Aérea.

Instale e carregue os pacotes `ggcal` e `ggplot2`.

```
library(ggplot2)
library(ggcal)
```

Defina uma paleta de cores em modo gradiente. Utilize o comando `scale_fill_gradient`. A cor inicial da paleta deve ser `#4575b4` e a cor final, `#d73027`. A paleta deve ser armazenada no objeto `pal`.

```
pal <- scale_fill_gradient(low = "#4575b4", high = "#d73027")
pal
```

```
<ScaleContinuous>
```

```
Range:
```

```
Limits:    0 --    1
```

Crie uma função chamada `baseCalendario` que recebe 2 argumentos a seguir: `stats` (tibble com resultados calculados na questão 4) e `cia` (sigla da Cia. Aérea de interesse). A função deverá: Criar um subconjunto de `stats` de forma a conter informações de atraso e data apenas da Cia. Aérea dada por `cia`. Para o subconjunto acima, montar a base do calendário, utilizando `ggcal(x, y)`. Nesta notação, `x` representa as datas de interesse e `y`, os percentuais de atraso para as datas descritas em `x`. Retornar para o usuário a base do calendário criada acima.

```
baseCalendario <- function(stats, cia) {
  subconjunto <- stats[stats$Cia == cia, c("Data", "Perc")]
  calendario <- ggcal(subconjunto$Data, subconjunto$Perc)
  return(calendario)
}
```

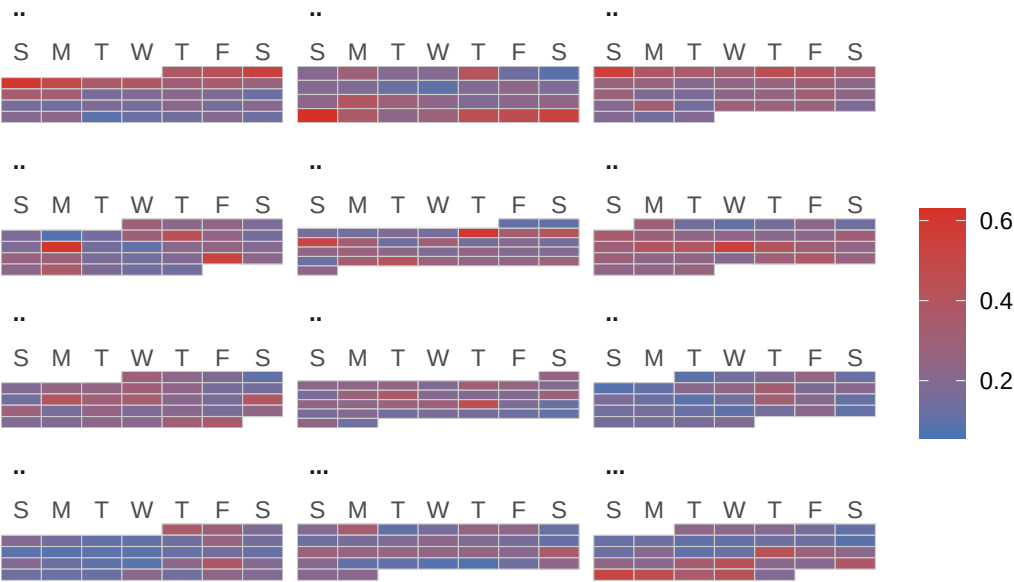
Executar a função `baseCalendario` para cada uma das Cias. Aéreas e armazenar os resultados, respectivamente, nas variáveis: `cAA`, `cDL`, `cUA` e `cUS`.

```
cAA <- baseCalendario(aleluia, "AA")
cDL <- baseCalendario(aleluia, "DL")
cUA <- baseCalendario(aleluia, "UA")
cUS <- baseCalendario(aleluia, "US")
```

Para cada uma das Cias. Aéreas, apresente o mapa de calor respectivo utilizando a combinação de camadas do `ggplot2`. Lembre-se de adicionar um título utilizando o comando `ggtitle`. Por exemplo, `cXX + pal + ggtitle("Título")`.

```
grafico_AA <- cAA + pal + ggtitle("American Airlines (AA)")
grafico_DL <- cDL + pal + ggtitle("Delta Air Lines (DL)")
grafico_UA <- cUA + pal + ggtitle("United Airlines (UA)")
grafico_US <- cUS + pal + ggtitle("US Airways (US)")
grafico_AA
```

American Airlines (AA)



grafico_DL

Delta Air Lines (DL)

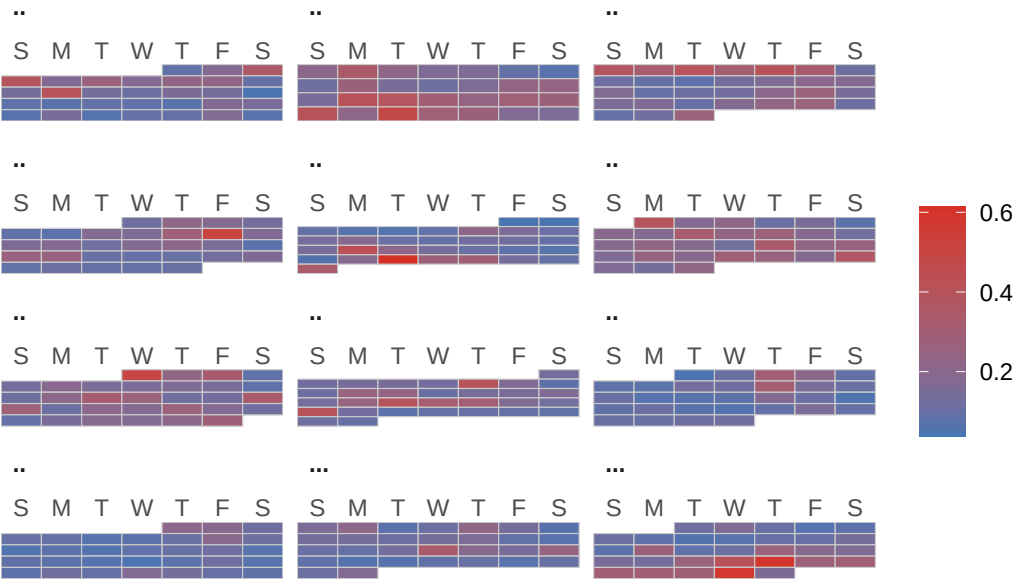
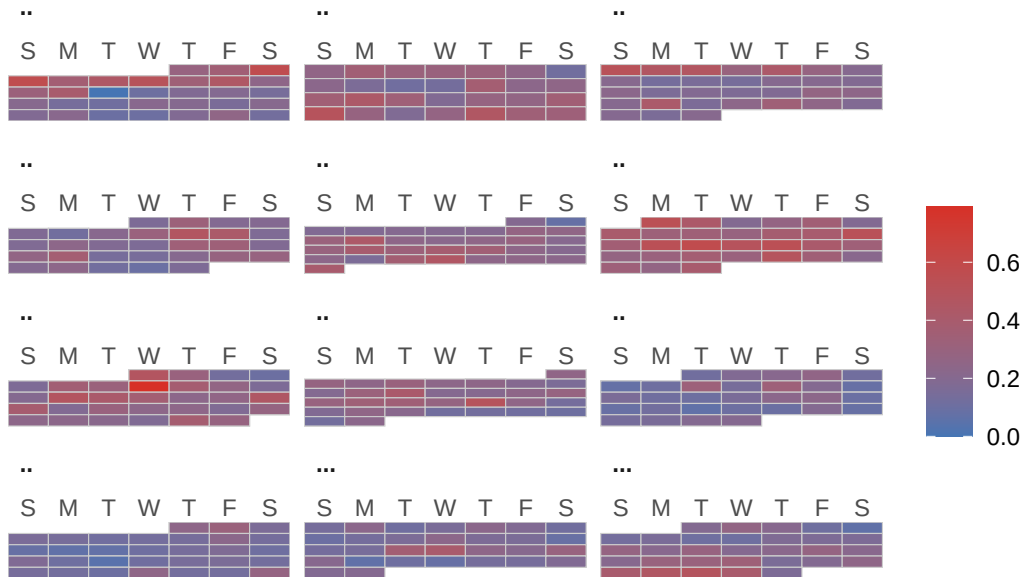


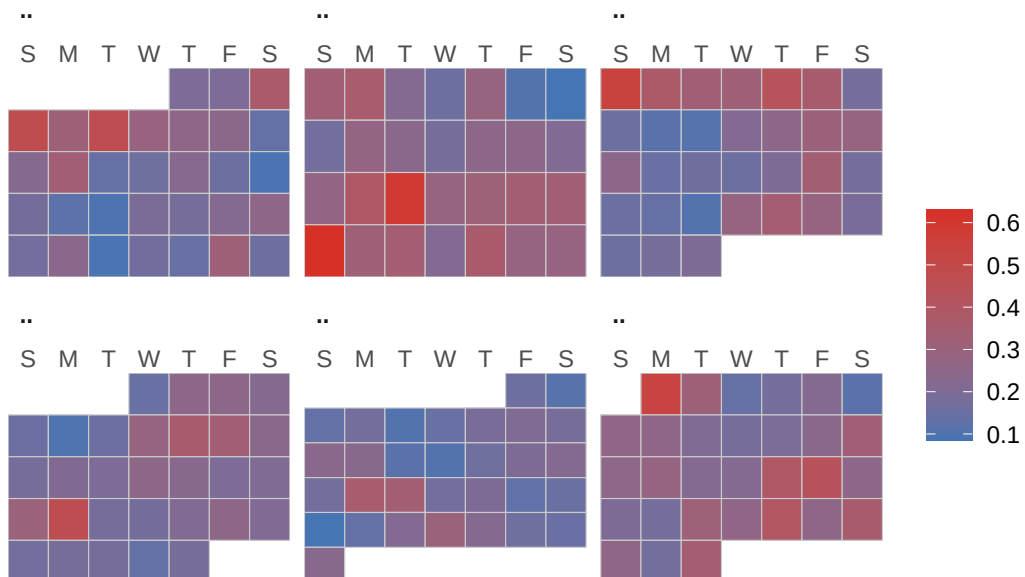
grafico-UA

United Airlines (UA)



grafico_US

US Airways (US)



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-24 23:13:36 -03"
```